[0x3d]

.loop

A: $rax_{i+1} = rax_i \oplus rcx_i$

$rcx_{i+1} = rax_i \wedge rcx_i$

B: $rdx_{i+1} = rdx_i \oplus rcx_{i+1}$

$rcx_{i+2} = rdx_i \wedge rcx_{i+1}$

ripple carry addition

$rcx_{i+2} = 0$? N → i+4; Y → EXIT

```
        mov     rcx, 1
.loop:
        xor     rax, rcx
        not     rax
        and     rcx, rax
        not     rax

        xor     rdx, rcx
        not     rdx
        and     rcx, rdx
        not     rdx
        shl     rcx, 1
        jnz     .loop
```

A: xor, not, and, not

B: xor, not, and, not

C: shl, jnz

which modify the ZFs??

$a^0 \to a^1$

for (i=0, ...)

measures how many shifts x overwrites itself?

$a^5\ a^4\ a^3\ a^2\ a^1\ a^0$

hit-by-bit masked zone sort

rcx

handles further multiple phases here ...

ie for loop:

$rax_{i-1} \oplus rcx_{i-1}$

$rax_i = rax_0 \oplus rcx_0$

$rcx = rcx_0 \wedge \neg(rax_0 \oplus rcx_0)$

$rdx = rcx_0 \wedge \neg(rax_0 \oplus rcx_0)$

$rcx = rcx_0 \wedge \neg(rcx_0 \wedge \neg(rax_0 \oplus rcx_0))$

[0x3d]

C: rcx <<1, jnz → $rcx = 0$? Y → EXIT; N → see result $\in 2^k$ & max for 2^k

```
mov rcx, 1
.loop:
```

A:
- xor $\boxed{rax}$, rcx
- not rax
- and rcx, rax
- not rax

ie rax[i] += rcx[i]

$$rax_{i+1} = \overbrace{rax_i \oplus rcx_i}^{rax_{i-1} \oplus rcx_{i-1}}$$

sum of the 2 bits

(of sum) carry out

$$rax_i \wedge rcx_i = rcx_{i+1}$$

$$rcx_{i+1} = \{\neg rax_{i+1} \wedge rcx_i\}$$

$$= \neg(rax_i \oplus rcx_i) \wedge rcx_i$$

$$= (rax_i \neg\oplus rcx_i) \wedge rcx_i$$

$$= [(rax_i \wedge rcx_i) \vee (\neg rax_i \wedge \neg rcx_i)] \wedge rcx_i$$

distributive law

$$= \underbrace{[(rax_i \wedge rcx_i) \wedge rcx_i]}_{\underbrace{rax_i \wedge (rcx_i \wedge rcx_i)}_{rcx_i}} \vee \underbrace{[(\neg rcx_i \wedge \neg rcx_i) \wedge rcx_i]}_{\underbrace{\neg rcx_i \wedge 0}_{0}}$$

$$\{(rax_i \wedge rcx_i) \quad \vee \quad 0\} \xrightarrow{Id} rax_i \wedge rcx_i$$

B:
- xor rdx, rcx
- not rdx
- and rcx, rdx
- not rdx

ie rdx[i] += carry out of rax

C:
- shl rcx, 1
- jnz .loop

ie rcx = 0 ?

Y ↓ EXIT

N ↓ continue

add carry out to rdx

$$rdx_{i+1} = rdx_i \oplus rcx_{i+1}$$

$$rcx = \cancel{\neg(rdx_i \oplus} \cancel{\neg rcx} \; rcx_{i+1} \wedge \neg rdx_{i+1}$$

$$= (rax_i \wedge rcx_i) \wedge \neg(rdx_i \oplus rcx_{i+1})$$

$$= (rax_i \wedge rcx_i) \wedge \neg(rdx_i \oplus (rax_i \wedge rcx_i)) \quad *$$

$$= \underbrace{rcx_{i+1} \wedge \neg(rdx_i \oplus rcx_{i+1})}$$

|| {logic above ie $\neg(a \oplus c) \wedge c \equiv a \wedge c$

$$= rax_i \wedge rcx_{i+1}$$

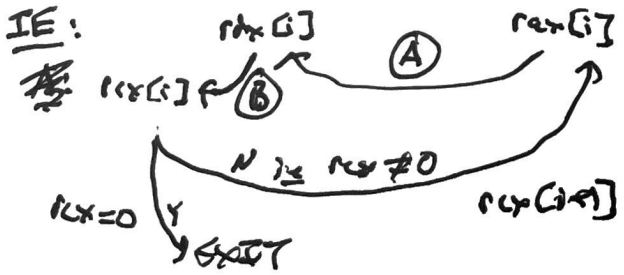
$$rcx_{i+2} = rdx_i \wedge (rax_i \wedge rcx_i)$$

add resulting carry (from) to rcx[i+2] for next loop

N

Y

∃ a resulting carry ???

ie rcx = carry out of rdx << 1

IE: